Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.:5

Team No.:7

Project Title: Linux Process Resource Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030376	V.VedaPriya	
252030386	B.Harini	
2520030443	K.Sai Samyuktha	

1. Abstract

Linux Process Resource Monitoring System is a Linux-based Operating Systems project designed to continuously monitor running processes and analyze their resource utilization. In a Linux system, multiple processes execute simultaneously, and some processes may consume excessive CPU or memory resources, become unresponsive, or continue running for an extended period. This project addresses the problem by providing a monitoring mechanism that collects process information and detects processes whose resource usage exceeds predefined limits.

The project focuses on key Operating Systems and system programming concepts such as process management, process states, process identification, resource monitoring, the Linux /proc filesystem, signal handling, and process control. Linux interfaces and system calls can be used to collect information such as Process ID (PID), process state, CPU usage, and memory usage. Signals can also be used to stop or terminate processes that exceed predefined resource thresholds.

The proposed solution consists of a Linux-based application developed in C/C++ that continuously reads process information from the /proc filesystem and monitors the resource usage of running processes. The system generates warnings when predefined CPU or memory limits are exceeded and maintains logs of process activity and resource utilization. It also provides an administrator-controlled mechanism for starting and stopping the monitoring process.

The expected outcome is a functional Linux process resource monitoring system that provides real-time information about running processes and their resource consumption. The project provides practical understanding of Linux process management, /proc interfaces, resource monitoring, signal-based process control, and system programming while strengthening the ability to develop efficient Linux-based monitoring applications.

2. Problem Statement

In a Linux operating system, multiple processes run simultaneously and compete for system resources such as CPU and memory. In systems running a large number of processes, some processes may consume excessive resources, become unresponsive, or continue executing indefinitely. Without continuous monitoring, such processes may negatively affect overall system performance and resource availability.

This project addresses the problem of monitoring running processes and identifying processes whose CPU or memory usage exceeds predefined limits. The system continuously collects process information and provides warnings when abnormal or excessive resource consumption is detected.

The project is relevant to Operating Systems and Systems Programming because it involves practical concepts such as process management, process states, process identification, resource utilization, Linux /proc filesystem interfaces, signal handling, and process control.

The proposed system uses Linux interfaces to obtain information about running processes and their resource usage. When a process exceeds a predefined resource threshold, the administrator can be notified and appropriate control actions, such as stopping or terminating the process, can be performed using Linux signals.

The project aims to demonstrate that continuous process monitoring and resource analysis can help administrators observe system behavior, identify resource-intensive processes, and manage problematic processes efficiently in a Linux environment.

3. Objectives

- To monitor running processes and their current states in a Linux system.
- To collect CPU and memory usage information of running processes.
- To read process-related information from the Linux **`/proc`** filesystem.
- To identify processes whose resource usage exceeds predefined CPU or memory limits.
- To generate warnings when resource thresholds are exceeded.
- To use Linux signals to stop or terminate problematic processes.
- To track process creation and termination during monitoring.
- To maintain logs of process activity and resource usage.
- To display current process and system resource information.
- To provide an administrator-controlled mechanism for starting and stopping the monitoring system.

4. Proposed Methodology

The project will be designed and implemented as a Linux-based C/C++ application that continuously monitors running processes and analyzes their CPU and memory resource utilization.

1. Process Information Collection:
The monitoring application will identify currently running processes and collect relevant information such as Process ID (PID), process state, CPU usage, and memory usage.

2. Reading the /proc Filesystem:
The system will access the Linux /proc filesystem to obtain process-related and system-related information. Each running process can be observed through its corresponding process information available in /proc.

3. Resource Monitoring:
The application will periodically monitor CPU and memory usage of running processes. The collected information will be displayed to provide an overview of current process activity and resource consumption.

4. Threshold Detection:
Predefined CPU and memory limits will be established. The system will compare the resource usage of each monitored process with these limits to identify processes consuming excessive resources.

5. Warning Generation:
When a process exceeds a predefined resource threshold, the system will generate an appropriate warning and record the event.

6. Process Control:
Problematic processes can be controlled using Linux signals. Depending on the administrator's decision, a process may be stopped or terminated.

7. Process Activity Tracking:
The monitoring system will track important process events, including process creation and termination, during the monitoring period.

8. Logging:
The system will maintain logs containing process activity, resource usage, warnings, and process-control actions.

9. Administrator Control:
The administrator will be provided with a mechanism to start and stop the monitoring process in a controlled manner.

10. Execution Flow:
The overall flow of the system will be:

Start Monitoring → Read Process Information from /proc → Collect CPU and Memory Usage → Compare with Resource Thresholds → Generate Warning if Required → Perform Process Control Action → Record Activity in Log → Update Process Information → Stop Monitoring.

11. Testing:
The application will be executed and tested in a Linux environment using different running processes and resource conditions. The output will be observed to verify process monitoring, threshold detection, warning generation, logging, and process control.

The methodology provides a practical demonstration of Linux process monitoring, resource management, /proc filesystem interfaces, signal-based process control, and system programming.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Monitoring – /proc Filesystem	Retrieves information about currently running processes for monitoring and analysis.

Process Identification – getpid(), getppid()	Retrieves the PID and PPID of processes for identification and management.
Directory Access – opendir(), readdir(), closedir()	Accesses the /proc directory and identifies currently running processes.
Process Information – /proc/[PID]/stat	Retrieves information such as process state and CPU-related statistics.
Memory Monitoring – /proc/[PID]/status	Retrieves memory-related information of processes for resource monitoring
Process Control – SIGSTOP, SIGCONT, SIGTERM	Demonstrates pausing, resuming, and safely terminating processes using Linux signals.
Signal Sending – kill()	Sends signals to processes that exceed predefined resource limits for process control.
Logging – open(), write(), close()	Records process activity, resource usage, warnings, and process-control actions.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030376	V.VedaPriya	Process monitoring and /proc information collection
2520030384	B.Harini	CPU and memory resource monitoring
2520030443	K.Sai Samyuktha	Threshold detection and warning generation
2520030376	V.VedaPriya	Signal-based process control
2520030443,2520030384	K.Sai Samyuktha,B.Harini	Testing, logging, and documentation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the process communication program.
C / C++	Used to implement process creation, signal handling, communication, and process control.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for signals, processes, and synchronization.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.

8. Expected Outcome

The project is expected to produce a functional Linux-based process resource monitoring application that demonstrates how running processes can be continuously monitored and analyzed based on their resource usage.

1. The program will successfully identify and monitor currently running processes.
2. Each monitored process will display information such as PID, process state, CPU usage, and memory usage.
3. The system will collect process information from the Linux /proc filesystem.
4. The application will detect processes that exceed predefined CPU or memory limits.
5. The system will generate warnings when resource thresholds are exceeded.
6. Problematic processes can be controlled using appropriate Linux signals.
7. The program will maintain logs of process activity and resource usage.
8. Overall, the project will provide practical understanding of Linux process management, resource monitoring, /proc interfaces, signal-based process control, and system programming.

9. GitHub Repository

GitHub Repository Link:

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: